Knight Moves - BFS in 2d Grid

The Knight Moves problem involves finding the shortest path a chess knight can take between two positions on a standard 8x8 chessboard. A knight moves in an L-shaped pattern, either two squares in one direction followed by one square perpendicular, or one square in one direction followed by two squares perpendicular, resulting in eight possible moves from any position unless constrained by the board edges. The input consists of multiple test cases where each case provides two chessboard squares in algebraic notation, with a lowercase letter (a-h) representing the column and a digit (1-8) representing the row, separated by a space. The output for each test case must be precisely formatted as "To get from [start] to [end] takes [n] knight moves." where [n] is the minimum number of moves required.

The solution employs a breadth-first search (BFS) algorithm because the chessboard can be modeled as an unweighted graph where each square is a node and knight moves create edges between them. BFS guarantees finding the shortest path in terms of the number of moves since all moves have equal cost. The algorithm starts from the initial square, explores all possible knight moves level by level, and stops when it reaches the destination square, counting the moves along the way. Key implementation details include converting algebraic notation to coordinate indices, validating moves to ensure they remain within the board boundaries, tracking visited squares to avoid cycles, and managing the BFS queue efficiently.

Sample cases illustrate various scenarios, such as short moves like b1 to c3 taking one move, medium moves like e2 to e4 taking two moves, long diagonal traversals like a1 to h8 taking six moves, and the edge case of identical start and end squares taking zero moves. The problem highlights the practical challenge of pathfinding in chess-based

problems while emphasizing the importance of correct output formatting and efficient algorithm choice for real-time performance.

Steps to Solve:

Step 1: Understand the Problem
We need to find the minimum number of knight moves required to go from one square to another on an 8×8 chessboard. A knight moves in an L-shape: two squares in one direction and one square perpendicular, giving eight possible moves from any position.

Step 2: Set Up the Chessboard Representation
Represent the chessboard as an 8×8 grid. Convert algebraic notation (like "e2") to coordinates: letters a-h become columns 0-7, numbers 1-8 become rows 0-7.

Step 3: Define Knight Moves
Create two arrays to represent all eight possible knight moves:

- dx[] = {2, 2, 1, 1, -1, -1, -2, -2}
- dy[] = {1, -1, 2, -2, 2, -2, 1, -1}

Step 4: Initialize BFS Components
Create a visited matrix to track explored squares, a distance matrix to store move counts, and a queue for BFS traversal. Use a structure/class to store position (x, y) and move count.

Step 5: Handle Special Case
If start and end positions are the same, return 0 moves immediately.

Step 6: Implement BFS Algorithm

- Enqueue the starting position with move count 0
- Mark starting position as visited
- While queue is not empty:
    - Dequeue current position

- For each of the eight possible knight moves:
  - Calculate new position
  - Check if new position is valid (within board) and not visited
  - If new position equals destination, return current moves + 1
  - Else mark as visited, set distance, and enqueue with moves + 1

Step 7: Process Input and Output

- Read input pairs until end of file
- For each pair, call BFS function
- Print results in required format: "To get from xx to yy takes n knight moves."

Step 8: Boundary Validation
Ensure all moves stay within the 8×8 board by checking that new coordinates satisfy: $0 \leq x < 8$ and $0 \leq y < 8$.

Input:

e2 e4

a1 b2

b2 c3

a1 h8

a1 h7

h8 a1

b1 c3

f6 f6

Execution:

Input: e2 e4

1. **Read Input:** start = "e2", end = "e4"
2. **Convert Coordinates:**
   - e2 → column 'e' = 4, row '2' = 1 → (4,1)
   - e4 → column 'e' = 4, row '4' = 3 → (4,3)
3. **Initialize BFS:**

- Create visited[8][8] = all false
- Queue = [(4,1,0)]
- visited[4][1] = true

4. **Process Queue:**
   - Dequeue (4,1,0)
   - Generate 8 knight moves: (6,2), (6,0), (5,3), (5,-1), (3,3), (3,-1), (2,2), (2,0)
   - Valid moves (within board): (6,2), (6,0), (5,3), (3,3), (2,2), (2,0)
   - Enqueue all valid moves with moves=1
5. **Continue BFS:**
   - Dequeue (6,2,1)
   - Generate moves: (8,3), (8,1), (7,4), (7,0), (5,4), (5,0), (4,3), (4,1)
   - Found destination (4,3) → return 1 + 1 = 2
6. **Output:** "To get from e2 to e4 takes 2 knight moves."

Input: a1 b2

1. **Read Input:** start = "a1", end = "b2"
2. **Convert Coordinates:** a1 → (0,0), b2 → (1,1)
3. **BFS Search:** Explores multiple paths through intermediate squares
4. **Find Path:** (0,0)→(2,1)→(1,3)→(3,2)→(1,1) = 4 moves
5. **Output:** "To get from a1 to b2 takes 4 knight moves."

Input: b2 c3

1. **Read Input:** start = "b2", end = "c3"
2. **Convert Coordinates:** b2 → (1,1), c3 → (2,2)
3. **BFS Search:** Finds path through one intermediate square
4. **Find Path:** (1,1)→(2,3)→(2,2) = 2 moves
5. **Output:** "To get from b2 to c3 takes 2 knight moves."

Input: a1 h8

1. **Read Input:** start = "a1", end = "h8"
2. **Convert Coordinates:** a1 → (0,0), h8 → (7,7)
3. **BFS Search:** Explores longest diagonal path
4. **Find Path:** Multiple 6-move paths exist across board
5. **Output:** "To get from a1 to h8 takes 6 knight moves."

Input: a1 h7

1. **Read Input:** start = "a1", end = "h7"

2. **Convert Coordinates:** a1 → (0,0), h7 → (7,6)
3. **BFS Search:** Slightly shorter than h8 path
4. **Find Path:** Optimal path found in 5 moves
5. **Output:** "To get from a1 to h7 takes 5 knight moves."

Input: h8 a1

1. **Read Input:** start = "h8", end = "a1"
2. **Convert Coordinates:** h8 → (7,7), a1 → (0,0)
3. **BFS Search:** Reverse of previous path
4. **Find Path:** Same 6 moves as forward direction
5. **Output:** "To get from h8 to a1 takes 6 knight moves."

Input: b1 c3

1. **Read Input:** start = "b1", end = "c3"
2. **Convert Coordinates:** b1 → (1,0), c3 → (2,2)
3. **BFS Search:** Direct L-shaped move available
4. **Find Path:** (1,0)→(2,2) = 1 move
5. **Output:** "To get from b1 to c3 takes 1 knight moves."

Input: f6 f6

1. **Read Input:** start = "f6", end = "f6"
2. **Convert Coordinates:** f6 → (5,5), f6 → (5,5)
3. **Special Case:** Same start and end position
4. **Immediate Return:** 0 moves without BFS
5. **Output:** "To get from f6 to f6 takes 0 knight moves."

Output:

To get from e2 to e4 takes 2 knight moves.

To get from a1 to b2 takes 4 knight moves.

To get from b2 to c3 takes 2 knight moves.

To get from a1 to h8 takes 6 knight moves.

To get from a1 to h7 takes 5 knight moves.

To get from h8 to a1 takes 6 knight moves.

To get from b1 to c3 takes 1 knight moves.

To get from f6 to f6 takes 0 knight moves.

Pseudocode:

```
STRUCT Position:
    x, y, moves

CONSTANT dx = [2, 2, 1, 1, -1, -1, -2, -2]
CONSTANT dy = [1, -1, 2, -2, 2, -2, 1, -1]

FUNCTION isValid(x, y):
    RETURN (x >= 0 AND x < 8 AND y >= 0 AND y < 8)

FUNCTION bfs(start, end):
    IF start == end:
        RETURN 0

    visited[8][8] = {false}

    startX = start[0] - 'a'
    startY = start[1] - '1'
    endX = end[0] - 'a'
    endY = end[1] - '1'

    queue = empty
    queue.push({startX,  startY, 0})
    visited[startX][startY] = true
```

```
    WHILE queue is not empty:
        current = queue.pop()

        FOR i = 0 to 7:
            newX = current.x + dx[i]
            newY = current.y + dy[i]

            IF isValid(newX, newY) AND NOT visited[newX][newY]:
                IF newX == endX AND newY == endY:
                    RETURN current.moves + 1

                visited[newX][newY] = true
                queue.push({newX,  newY, current.moves + 1})

    RETURN -1

FUNCTION main():
    WHILE read start AND read end:
        moves = bfs(start, end)
        PRINT "To get from " + start + " to " + end + " takes " + moves + " knight moves."
```